COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/2026	**DATE SUBMITTED:**	10/20/2026
NAME:	Libo-on, Gian Carlo G. Antonio, Christine Mae P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **ALGORITHMIC STRATEGIES**

1. PROCEDURE OUTPUT

Part 1:

Screenshot

```
#include <iostream>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

    return 0;
}
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

The print function takes integer and string values as arguments. The thread class creates a new thread “t1” which executes the print function, followed by the values for its arguments. The join() function ensures that the thread is complete before the program can proceed to execution.

Table 13-1. Simple One-Threaded Example

Screenshot

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

```
2 : Computer1 : Competent
3 : Engineers
0 : T.I.P.
```

Analysis

This demonstrates multithreading. “std::vector<std::string> s” defines four strings, while “std::vector<std::thread> threads” is used to manage the threads. The threads.push_back() creates a new background thread for every string. “th.join()” waits for the threads to finish before the program terminates. The output is unpredictable or returns the strings in random order.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Write a definition of multithreading and its advantages/disadvantages.

Antonio	Libo-on
Multithreading lets the program implement concurrency and execute multiple tasks efficiently. It improves performance and makes better use of CPU resources. However, it can be unpredictable, making debugging more difficult.	Multithreading allows you to do tasks simultaneously with minimal disk usage. A multithreading program consists of many small independent units that run concurrently. Multithreading allows you to save CPU utilization and increase storage efficiency since it helps you consume less storage when the program is active. However, a multithreading code is hard to implement especially since it has an increased difficulty and bugs may occur unintentionally.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Antonio	Libo-on
Multithreading can be used in gaming to handle graphics, sound, and logic. It can also be used in file processing by reading and processing multiple files simultaneously. A web server can utilize multithreading to handle multiple client requests at the same time.	Multithreading has a lot of uses. It decreases response time and increases system efficiency by allowing multitasking. Most computer systems and social media implements multithreading. For example, web servers use multithreading to handle many tasks at once. It uses multithreading to handle user requests. Not being able to handle it properly leads to web traffic. Another example is the operating system itself. OS uses multithreading to allow you to handle multiple tasks. Without multithreading an OS wouldn't be able to h=

3. Differentiate between parallelism and concurrency.

Antonio	Libo-on
Concurrency can manage multiple threads simultaneously by switching them on a single CPU core. Parallelism functions similarly, although it executes tasks and uses multiple CPU cores.	Concurrency allows you to do multiple things simultaneously using a single cpu core. On the other hand, parallelism allows you to do multiple processes simultaneously using many cpu cores.

Part B:

Code & Output	

```cpp
#include <iostream>
#include <thread>
#include <string>
#include <mutex>
#include <condition_variable>

int globalValue = 0;
std::mutex mtx;
std::condition_variable cv;


int activeStep = 0;

void add(int value, int myStep) {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [myStep] { return activeStep ==
myStep; });

    globalValue += value;

    activeStep = 0;
    cv.notify_all();
}

void display(const std::string& label) {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << label << ": globalValue = " <<
globalValue << std::endl;
}

void triggerStep(int step) {
    {
        std::lock_guard<std::mutex> lock(mtx);
        activeStep = step;
    }
```

```
    cv.notify_all();
}

int main() {
    std::thread T1(add, 10, 1);
    std::thread T2(add, 20, 2);
    std::thread T3(add, 30, 3);

    display("Display 1 ");

    triggerStep(1);
    T1.join();
    display("Display 2 ");

    triggerStep(2);
    T2.join();
    display("Display 3 ");

    triggerStep(3);
    T3.join();
    display("Display 4 ");

    return 0;
}
```

```
Display 1 : globalValue = 0
Display 2 : globalValue = 10
Display 3 : globalValue = 30
Display 4 : globalValue = 60
```

Analysis

The code snippet shows a complex implementation of a simple thread. This happened because the threads initiated so fast, they were not functioning properly causing an output of 0,60,60,60. With this, I made the threads work properly. Based on this code snippet, the threads are initialized all at the same time and the joining works sequentially to print 0, 10, 30, 60.

Part C:

Code & Output	**main.cpp**

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include <chrono>
#include <iomanip>

#include "node.h"
#include "searchingAlgo.h"

const int DATA_SIZE = 1000000;


int main() {
    srand(static_cast<unsigned int>(time(0)));

    int* dataset = new int[DATA_SIZE];
    for (int i = 0; i < DATA_SIZE; i++) {
        dataset[i] = rand() % 1000000;
    }

    std::sort(dataset, dataset + DATA_SIZE);

    int target = -1;

    std::cout << "Searching Algorithm Comparison (" <<
DATA_SIZE << " Elements)\n";
    std::cout << "Target: " << target << "\n";

    int linearOps = 0;
    int binaryOps = 0;

    auto startLinear =
std::chrono::high_resolution_clock::now();
    linearSearch(DATA_SIZE, target, dataset, linearOps);
```

```
    auto endLinear =
std::chrono::high_resolution_clock::now();
    auto timeLinear =
std::chrono::duration_cast<std::chrono::nanoseconds>(end
Linear - startLinear).count();


    auto startBinary =
std::chrono::high_resolution_clock::now();
    binarySearchArr(dataset, DATA_SIZE, target,
binaryOps);
    auto endBinary =
std::chrono::high_resolution_clock::now();
    auto timeBinary =
std::chrono::duration_cast<std::chrono::nanoseconds>(end
Binary - startBinary).count();

    // --- RESULTS TABLE ---
    std::cout << std::left << std::setw(18) <<
"Algorithm"
              << std::setw(18) << "Comparisons"
              << "Execution Time\n";
    std::cout << std::left << std::setw(18) << "Linear
Search"
              << std::setw(18) << linearOps
              << timeLinear << " ns\n";
    std::cout << std::left << std::setw(18) << "Binary
Search"
              << std::setw(18) << binaryOps
              << timeBinary << " ns\n";

    delete[] dataset;
    return 0;
}
```

Analysis	**This is a linear and binary search multithreading with time accuracy and worst-time complexity. The multithread is applied when both linear and binary searching algorithms were executed at the same time by calling a single execution process.**

	This allowed the cpu or the program to run both the binary and linear searching algo at the same time to look for the worst time complexity of both linear and binary search algorithm.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Helping me in fixing task 2 because it doesn't work properly**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.